

# Código fuente completo para armar los videos de NotebookLM

Este PDF junta el código real de todos los archivos que se usan en los 6 videos definidos en `notebook.md`. Está organizado en 6 secciones ("VIDEO 1" a "VIDEO 6"), cada una con el nombre de cada archivo seguido de su código completo, tal cual está en el proyecto. Subí este mismo PDF como fuente única en NotebookLM para los 6 videos — en el prompt de cada uno indicá "mirá el apartado de VIDEO N" para que se enfoque solo en esa sección.

## VIDEO 1 — Config de Android/Gradle

android/app/build.gradle.kts

```
plugins {  
    id("com.android.application")  
    // The Flutter Gradle Plugin must be applied after the Android and Kotlin Gradle plugins.  
    id("dev.flutter.flutter-gradle-plugin")  
}  
  
android {  
    namespace = "com.cowcontrol.app_cowcontrol"  
    compileSdk = flutter.compileSdkVersion  
    ndkVersion = flutter.ndkVersion  
  
    compileOptions {  
        sourceCompatibility = JavaVersion.VERSION_17  
        targetCompatibility = JavaVersion.VERSION_17  
    }  
  
    defaultConfig {  
        applicationId = "com.cowcontrol.app_cowcontrol"  
        minSdk = flutter.minSdkVersion  
        targetSdk = flutter.targetSdkVersion  
        versionCode = flutter.versionCode  
        versionName = flutter.versionName  
    }  
  
    buildTypes {  
        release {  
            // Signing with the debug keys for now, so `flutter run --release` works.  
            signingConfig = signingConfigs.getByName("debug")  
        }  
    }  
}  
  
kotlin {  
    compilerOptions {  
        jvmTarget = org.jetbrains.kotlin.gradle.dsl.JvmTarget.JVM_17  
    }  
}  
  
flutter {  
    source = "../.."  
}
```

## android/build.gradle.kts

```
allprojects {
    repositories {
        google()
        mavenCentral()
    }
}

val newBuildDir: Directory =
    rootProject.layout.buildDirectory
        .dir(".././build")
        .get()
rootProject.layout.buildDirectory.value(newBuildDir)

subprojects {
    val newSubprojectBuildDir: Directory = newBuildDir.dir(project.name)
    project.layout.buildDirectory.value(newSubprojectBuildDir)
}
subprojects {
    project.evaluationDependsOn(":app")
}

tasks.register<Delete>("clean") {
    delete(rootProject.layout.buildDirectory)
}
```

## android/settings.gradle.kts

```
pluginManagement {
    val flutterSdkPath =
        run {
            val properties = java.util.Properties()
            file("local.properties").inputStream().use { properties.load(it) }
            val flutterSdkPath = properties.getProperty("flutter.sdk")
            require(flutterSdkPath != null) { "flutter.sdk not set in local.properties" }
            flutterSdkPath
        }

    includeBuild("$flutterSdkPath/packages/flutter_tools/gradle")

    repositories {
        google()
        mavenCentral()
        gradlePluginPortal()
    }
}

plugins {
    id("dev.flutter.flutter-plugin-loader") version "1.0.0"
    id("com.android.application") version "9.0.1" apply false
    id("org.jetbrains.kotlin.android") version "2.3.20" apply false
}

include(":app")
```

## android/gradle.properties

```
org.gradle.jvmargs=-Xmx8G -XX:MaxMetaspaceSize=4G -XX:ReservedCodeCacheSize=512m -XX:+HeapDumpOnOutOfMemoryError
android.useAndroidX=true
# This newDsl flag was added by the Flutter template
android.newDsl=false
# This builtInKotlin flag was added by the Flutter template
android.builtInKotlin=false
```

## android/app/src/main/AndroidManifest.xml

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <!-- Sin esto, el build de RELEASE no tiene acceso a internet (el permiso
    de android/app/src/debug/AndroidManifest.xml solo se mezcla en debug
    y profile, nunca en release) - la app no podía hablar con Supabase
    en ningún APK instalado fuera de "flutter run". -->
  <uses-permission android:name="android.permission.INTERNET"/>
  <application
    android:label="CowControl"
    android:name="${applicationName}"
    android:icon="@mipmap/ic_launcher">
    <activity
      android:name=".MainActivity"
      android:exported="true"
      android:launchMode="singleTop"
      android:taskAffinity=""
      android:theme="@style/LaunchTheme"

      android:configChanges="orientation|keyboardHidden|keyboard|screenSize|smallestScreenSize|locale|layoutDirection|fontScale|screenLayout|density|uiMode"
      android:hardwareAccelerated="true"
      android:windowSoftInputMode="adjustResize">
      <meta-data
        android:name="io.flutter.embedding.android.NormalTheme"
        android:resource="@style/NormalTheme"
      />
      <intent-filter>
        <action android:name="android.intent.action.MAIN"/>
        <category android:name="android.intent.category.LAUNCHER"/>
      </intent-filter>
    </activity>
    <meta-data
      android:name="flutterEmbedding"
      android:value="2" />
  </application>
  <queries>
    <intent>
      <action android:name="android.intent.action.PROCESS_TEXT"/>
      <data android:mimeType="text/plain"/>
    </intent>
  </queries>
</manifest>
```

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <!-- Theme applied to the Android Window while the process is starting -->
    <style name="LaunchTheme" parent="@android:style/Theme.Light.NoTitleBar">
        <item name="android:windowBackground">@drawable/launch_background</item>
    </style>
    <!-- Theme applied to the Android Window as soon as the process has started -->
    <style name="NormalTheme" parent="@android:style/Theme.Light.NoTitleBar">
        <item name="android:windowBackground">?android:colorBackground</item>
    </style>
</resources>
```

## VIDEO 2 — Config raíz del proyecto Flutter

### pubspec.yaml

```
name: app_cowcontrol
description: "A new Flutter project."
publish_to: 'none'

version: 1.0.0+1

environment:
  sdk: ^3.12.2

dependencies:
  flutter:
    sdk: flutter

  cupertino_icons: ^1.0.8
  supabase_flutter: ^2.16.0
  flutter_dotenv: ^6.0.1

dev_dependencies:
  flutter_test:
    sdk: flutter

  flutter_lints: ^6.0.0

# Genera el icono de la app (mipmaps de Android + iOS) a partir de un solo
# PNG fuente – se corre una vez con `dart run flutter_launcher_icons`.
flutter_launcher_icons: ^0.14.4

# Configuración de flutter_launcher_icons: logo oficial (cabeza de vaca +
# caravana RFID) extraído de la Carpeta técnica, fondo blanco.
flutter_launcher_icons:
  android: true
  ios: true
  image_path: "assets/icon/icon.png"
  adaptive_icon_background: "#FFFFFF"
  adaptive_icon_foreground: "assets/icon/icon_foreground.png"

flutter:
  uses-material-design: true

# .env guarda SUPABASE_URL y SUPABASE_ANON_KEY; flutter_dotenv lo lee como asset.
assets:
  - .env
  - assets/images/
```

## pubspec.lock (primeras líneas – es un archivo largo autogenerado)

```
# Generated by pub
# See https://dart.dev/tools/pub/glossary#lockfile
packages:
  app_links:
    dependency: transitive
    description:
      name: app_links
      sha256: f8db46d2ea9ff6f3a37191a7fd5b7813da1253ace8c32c1b9eadace41d1188ea
      url: "https://pub.dev"
    source: hosted
    version: "7.2.1"
  app_links_linux:
    dependency: transitive
    description:
      name: app_links_linux
      sha256: f5f7173a78609f3dfd4c2ff2c95bd559ab43c80a87dc6a095921d96c05688c81
      url: "https://pub.dev"
    source: hosted
    version: "1.0.3"
# ... y así sigue, un bloque como este por cada paquete que pubspec.yaml
# termina necesitando (incluye dependencias indirectas, no solo las que
# vos declaraste a mano) – nunca se edita este archivo con la mano.
```

## **.gitignore**

```
# Miscellaneous
*.class
*.log
*.pyc
*.swp
.DS_Store
.atom/
.build/
.buildlog/
.history
.svn/
.swiftpm/
migrate_working_dir/

# IntelliJ related
*.iml
*.ipr
*.iws
.idea/

# Flutter/Dart/Pub related
**/doc/api/
**/ios/Flutter/.last_build_id
.dart_tool/
.flutter-plugins-dependencies
.pub-cache/
.pub/
/build/
/coverage/

# Symbolication related
app.*.symbols

# Obfuscation related
app.*.map.json

# Android Studio will place build artifacts here
/android/app/debug
/android/app/profile
/android/app/release

# Credenciales de Supabase – nunca se suben al repo
.env

# APK compilado – binario pesado, se regenera solo, no debe crecer el historial de git
/apk_para_probar/
```

## **analysis\_options.yaml**

```
# This file configures the analyzer, which statically analyzes Dart code to
# check for errors, warnings, and lints.

include: package:flutter_lints/flutter.yaml

linter:
  rules:
    # avoid_print: false # Uncomment to disable the `avoid_print` rule
    # prefer_single_quotes: true # Uncomment to enable the `prefer_single_quotes` rule
```

## **.metadata**

```
# This file tracks properties of this Flutter project.
# Used by Flutter tool to assess capabilities and perform upgrades etc.
# This file should be version controlled and should not be manually edited.

version:
  revision: "ee80f08bbf97172ec030b8751ceab557177a34a6"
  channel: "stable"

project_type: app

# Tracks metadata for the flutter migrate command – lista todas las
# plataformas (android, ios, linux, macos, web, windows) que Flutter dejo
# preparadas desde el principio, se use esa plataforma o no.
```

## **README.md**

```
# app_cowcontrol

A new Flutter project.

## Getting Started

This project is a starting point for a Flutter application.

A few resources to get you started if this is your first Flutter project:

- [Learn Flutter](https://docs.flutter.dev/get-started/learn-flutter)
- [Write your first Flutter app](https://docs.flutter.dev/get-started/codelab)
- [Flutter learning resources](https://docs.flutter.dev/reference/learning-resources)
```

## VIDEO 3 — Las librerías del proyecto (pubspec.yaml a fondo)

### pubspec.yaml

```
name: app_cowcontrol
description: "A new Flutter project."
publish_to: 'none'

version: 1.0.0+1

environment:
  sdk: ^3.12.2

dependencies:
  flutter:
    sdk: flutter

  cupertino_icons: ^1.0.8
  supabase_flutter: ^2.16.0
  flutter_dotenv: ^6.0.1

dev_dependencies:
  flutter_test:
    sdk: flutter

  flutter_lints: ^6.0.0
  flutter_launcher_icons: ^0.14.4

flutter_launcher_icons:
  android: true
  ios: true
  image_path: "assets/icon/icon.png"
  adaptive_icon_background: "#FFFFFF"
  adaptive_icon_foreground: "assets/icon/icon_foreground.png"

flutter:
  uses-material-design: true
  assets:
    - .env
    - assets/images/
```

## lib/main.dart (ejemplo real de uso de las librerías)

```
import 'package:flutter/material.dart';
import 'package:flutter_dotenv/flutter_dotenv.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

import 'screens/login_screen.dart';
import 'screens/selector_campo_screen.dart';
import 'theme/app_theme.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  // Credenciales de Supabase: vienen del .env (nunca hardcodeadas ni subidas a git).
  await dotenv.load();
  await Supabase.initialize(
    url: dotenv.env['SUPABASE_URL']!,
    publishableKey: dotenv.env['SUPABASE_ANON_KEY']!,
  );

  runApp(const CowControlApp());
}

class CowControlApp extends StatelessWidget {
  const CowControlApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'CowControl',
      debugShowCheckedModeBanner: false,
      theme: AppTheme.tema,
      home: const AuthGate(),
    );
  }
}

/// Decide que pantalla mostrar segun haya o no una sesion activa de Supabase
/// Auth. Escucha onAuthStateChange, así que un login/logout desde cualquier
/// pantalla se refleja aca sin tener que navegar a mano.
class AuthGate extends StatelessWidget {
  const AuthGate({super.key});

  @override
  Widget build(BuildContext context) {
    return StreamBuilder<AuthState>(
      stream: Supabase.instance.client.auth.onAuthStateChange,
      builder: (context, snapshot) {
        final sesion =
          snapshot.data?.session ?? Supabase.instance.client.auth.currentSession;
        if (sesion == null) {
          return const LoginScreen();
        }
        return const SelectorCampoScreen();
      },
    );
  }
}
```

## VIDEO 4 — lib/models/ (las 4 clases de datos)

lib/models/animal.dart

```
/// Representa una fila de datos_animales. Solo incluye los campos que muestra la v1
/// de la app (la columna "sexo" se usa como categoria) – el resto de lo que trae el
/// mockup original (nombre, edad, maternidad, etc.) vive libre en observaciones.
class Animal {
  final String id;
  final String rfidUid;
  final String? categoria; // vaca / toro / ternero / ternera / vaquillona
  final String estadoVital; // vivo / muerto
  final String? fertilidad; // 'buena' / 'regular' / 'mala'
  final String? observaciones;
  final String campoId;

  const Animal({
    required this.id,
    required this.rfidUid,
    this.categoria,
    required this.estadoVital,
    this.fertilidad,
    this.observaciones,
    required this.campoId,
  });

  factory Animal.fromMap(Map<String, dynamic> map) {
    return Animal(
      id: map['id'] as String,
      rfidUid: map['rfid_uid'] as String,
      categoria: map['sexo'] as String?,
      estadoVital: map['estado_vital'] as String? ?? 'vivo',
      fertilidad: map['fertilidad'] as String?,
      observaciones: map['observaciones'] as String?,
      campoId: map['campo_id'] as String,
    );
  }
}
```

## lib/models/campo.dart

```
/// Representa una fila de datos_campo – un establecimiento ganadero del productor logueado.
class Campo {
  final String id;
  final String nombreCampo;
  final String? ubicacion;

  const Campo({
    required this.id,
    required this.nombreCampo,
    this.ubicacion,
  });

  factory Campo.fromMap(Map<String, dynamic> map) {
    return Campo(
      id: map['id'] as String,
      nombreCampo: map['nombre_campo'] as String,
      ubicacion: map['ubicacion'] as String?,
    );
  }
}
```

## lib/models/lectura.dart

```
/// Representa una fila de datos_lectura – un pasaje de un tag por el lector RFID/NFC.
class Lectura {
  final String id;
  final String rfidUid;
  final String? campoId;
  final DateTime fechaHora;
  final String? observaciones;

  const Lectura({
    required this.id,
    required this.rfidUid,
    this.campoId,
    required this.fechaHora,
    this.observaciones,
  });

  factory Lectura.fromMap(Map<String, dynamic> map) {
    return Lectura(
      id: map['id'] as String,
      rfidUid: map['rfid_uid'] as String,
      campoId: map['campo_id'] as String?,
      fechaHora: DateTime.parse(map['fecha_hora'] as String),
      observaciones: map['observaciones'] as String?,
    );
  }
}
```

```
/// Representa una fila de datos_vacuna. La app solo pide la mas reciente por
/// animal (fecha_vacuna descendente) para el resumen de la Pantalla 4.
class Vacuna {
  final String idAnimal;
  final String nombreVacuna;
  final DateTime? fechaVacuna;

  const Vacuna({
    required this.idAnimal,
    required this.nombreVacuna,
    this.fechaVacuna,
  });

  factory Vacuna.fromMap(Map<String, dynamic> map) {
    return Vacuna(
      idAnimal: map['id_animal'] as String,
      nombreVacuna: map['nombre_vacuna'] as String,
      fechaVacuna: map['fecha_vacuna'] != null
        ? DateTime.parse(map['fecha_vacuna'] as String)
        : null,
    );
  }
}
```

## VIDEO 5 — lib/services/ (la capa que habla con Supabase)

### lib/services/supabase\_service.dart

```
import 'package:supabase_flutter/supabase_flutter.dart';

/// Cliente unico de Supabase. Todos los demas servicios pasan por este getter
/// en vez de llamar a Supabase.instance.client directamente en cada archivo.
class SupabaseService {
  static SupabaseClient get client => Supabase.instance.client;
}
```

### lib/services/auth\_service.dart

```
import 'package:supabase_flutter/supabase_flutter.dart';

import 'supabase_service.dart';

/// Login, registro y cierre de sesion via Supabase Auth (Pantalla 0).
class AuthService {
  final _client = SupabaseService.client;

  User? get usuarioActual => _client.auth.currentUser;

  /// Se dispara cada vez que cambia el estado de sesion (login, logout, token
  /// renovado). El widget raiz (main.dart) escucha esto para decidir que pantalla mostrar.
  Stream<AuthState> get cambiosDeSesion => _client.auth.onAuthStateChange;

  Future<AuthResponse> iniciarSesion({
    required String email,
    required String password,
  }) {
    return _client.auth.signInWithPassword(email: email, password: password);
  }

  Future<AuthResponse> registrarse({
    required String email,
    required String password,
  }) {
    return _client.auth.signUp(email: email, password: password);
  }

  Future<void> cerrarSesion() {
    return _client.auth.signOut();
  }

  /// Manda el mail de "restablecer contraseña". El link del mail abre la
  /// pagina de GitHub Pages (docs/restablecer.html), no la app – ahi el
  /// usuario pone la contraseña nueva sin necesitar la vieja.
  Future<void> restablecerPassword({required String email}) {
    return _client.auth.resetPasswordForEmail(
      email,
      redirectTo: 'https://lovelyx-del.github.io/cowcontrol/restablecer.html',
    );
  }
}
```

## lib/services/campos\_service.dart

```
import '../models/campo.dart';
import 'supabase_service.dart';

/// Trae y crea campos (establecimientos) del productor logueado (Pantalla 1).
/// RLS ya filtra por productor_id = auth.uid() en el SELECT, no hace falta
/// pasarlo a mano; en el INSERT si hay que setearlo para cumplir la politica.
class CamposService {
  final _client = SupabaseService.client;

  Future<List<Campo>> traerCampos() async {
    final data = await _client.from('datos_campo').select().order('nombre_campo');
    return (data as List)
      .map((fila) => Campo.fromMap(fila as Map<String, dynamic>))
      .toList();
  }

  Future<Campo> crearCampo({required String nombreCampo, String? ubicacion}) async {
    final productorId = _client.auth.currentUser!.id;
    final fila = await _client
      .from('datos_campo')
      .insert({
        'nombre_campo': nombreCampo,
        'ubicacion': ubicacion,
        'productor_id': productorId,
      })
      .select()
      .single();
    return Campo.fromMap(fila);
  }
}
```

## lib/services/animales\_service.dart

```
import '../models/animal.dart';
import 'supabase_service.dart';

/// Trae los animales vivos de un campo (Pantallas 2 y 3) y el detalle de uno
/// en particular (Pantalla 4).
class AnimalesService {
  final _client = SupabaseService.client;

  Future<List<Animal>> traerAnimalesDeCampo(String campoId) async {
    final data = await _client
      .from('datos_animales')
      .select()
      .eq('campo_id', campoId)
      .eq('estado_vital', 'vivo')
      .order('rfid_uid');
    return (data as List)
      .map((fila) => Animal.fromMap(fila as Map<String, dynamic>))
      .toList();
  }

  Future<Animal> traerDetalle(String animalId) async {
    final fila = await _client.from('datos_animales').select().eq('id', animalId).single();
    return Animal.fromMap(fila);
  }

  Future<void> actualizarAnimal({
    required String animalId,
    String? observaciones,
    String? fertilidad,
  }) async {
    await _client.from('datos_animales').update({
      'observaciones': observaciones,
      'fertilidad': fertilidad,
    }).eq('id', animalId);
  }
}
```

## lib/services/lecturas\_service.dart

```
import 'package:supabase_flutter/supabase_flutter.dart';

import '../models/lectura.dart';
import 'supabase_service.dart';

/// Historial de lecturas (Pantalla 5) + suscripcion Realtime para el conteo
/// en vivo (Pantalla 3).
class LecturasService {
  final _client = SupabaseService.client;

  Future<List<Lectura>> traerHistorial(String campoId, {int limite = 50}) async {
    final data = await _client
      .from('datos_lectura')
      .select()
      .eq('campo_id', campoId)
      .order('fecha_hora', ascending: false)
      .limit(limite);
    return (data as List)
      .map((fila) => Lectura.fromMap(fila as Map<String, dynamic>))
      .toList();
  }

  /// Se suscribe a los inserts nuevos de datos_lectura para el campo dado.
  /// Supabase Realtime avisa por websocket apenas Postgres registra la fila
  /// (sin que la app tenga que preguntar "¿hay algo nuevo?" cada tanto).
  RealtimeChannel escucharNuevasLecturas({
    required String campoId,
    required void Function(Lectura) alRecibirLectura,
  }) {
    final canal = _client.channel('lecturas_campo_${campoId}');
    canal
      .onPostgresChanges(
        event: PostgresChangeEvent.insert,
        schema: 'public',
        table: 'datos_lectura',
        filter: PostgresChangeFilter(
          type: PostgresChangeFilterType.eq,
          column: 'campo_id',
          value: campoId,
        ),
        callback: (payload) {
          alRecibirLectura(Lectura.fromMap(payload.newRecord));
        },
      )
      .subscribe();
    return canal;
  }

  Future<void> dejarDeEscuchar(RealtimeChannel canal) {
    return _client.removeChannel(canal);
  }
}
```

## lib/services/vacunas\_service.dart

```
import '../models/vacuna.dart';
import 'supabase_service.dart';

/// Trae solo la vacuna mas reciente de un animal, para el resumen de la Pantalla 4
/// ("Ultima vacuna: Aftosa – 12/03/2026"), no el historial completo.
class VacunasService {
  final _client = SupabaseService.client;

  Future<Vacuna?> traerUltimaVacuna(String animalId) async {
    final data = await _client
      .from('datos_vacuna')
      .select()
      .eq('id_animal', animalId)
      .order('fecha_vacuna', ascending: false)
      .limit(1);
    final filas = data as List;
    if (filas.isEmpty) return null;
    return Vacuna.fromMap(filas.first as Map<String, dynamic>);
  }

  Future<void> agregarVacuna({
    required String idAnimal,
    required String nombreVacuna,
    DateTime? fechaVacuna,
  }) async {
    await _client.from('datos_vacuna').insert({
      'id_animal': idAnimal,
      'nombre_vacuna': nombreVacuna,
      if (fechaVacuna != null) 'fecha_vacuna': fechaVacuna.toIso8601String(),
    });
  }
}
```

## lib/services/conteos\_service.dart

```
import 'supabase_service.dart';

/// Maneja la fila de `conteos` de este conteo en vivo – la Raspberry Pi
/// consulta esa tabla para saber a que conteo con estado "en_curso" mandarle
/// las lecturas que va leyendo.
class ConteosService {
  final _client = SupabaseService.client;

  /// Si ya hay un conteo en pausa para este campo, lo reanuda. Si no hay
  /// ninguno, crea uno nuevo en estado "en_curso". Devuelve el id del conteo.
  Future<String> iniciarOReanudar(String campoId) async {
    final pausado = await _client
      .from('conteos')
      .select('id')
      .eq('campo_id', campoId)
      .eq('estado', 'pausado')
      .maybeSingle();

    if (pausado != null) {
      final id = pausado['id'] as String;
      await reanudar(id);
      return id;
    }

    final nuevo = await _client
      .from('conteos')
      .insert({
        'campo_id': campoId,
        'fecha': DateTime.now().toUtc().toIso8601String(),
        'estado': 'en_curso',
      })
      .select('id')
      .single();
    return nuevo['id'] as String;
  }

  Future<void> pausar(String conteoId) {
    return _client.from('conteos').update({'estado': 'pausado'}).eq('id', conteoId);
  }

  Future<void> reanudar(String conteoId) {
    return _client.from('conteos').update({'estado': 'en_curso'}).eq('id', conteoId);
  }

  Future<void> finalizar(String conteoId) {
    return _client.from('conteos').update({'estado': 'finalizado'}).eq('id', conteoId);
  }
}
```

## VIDEO 6 — lib/screens/ (panorama de las pantallas)

### lib/screens/login\_screen.dart (Pantalla 0)

```
import 'package:flutter/material.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

import '../services/auth_service.dart';
import '../theme/app_theme.dart';

/// Traduce los codigos de error de Supabase Auth a mensajes en español que un
/// productor pueda entender, sin exponer el detalle tecnico de la excepcion.
String _mensajeError(Object error, {required String mensajeGenerico}) {
  if (error is AuthException) {
    switch (error.code) {
      case 'email_not_confirmed':
        return 'Todavía no confirmaste tu email. Revisa tu correo y toca el '
          'link de confirmacion antes de iniciar sesion.';
      case 'invalid_credentials':
        return 'Email o contraseña incorrectos.';
      case 'user_already_exists':
        return 'Ya existe una cuenta registrada con ese email.';
      case 'weak_password':
        return 'La contraseña no cumple los requisitos minimos (8 caracteres, '
          'con letras y numeros).';
      case 'over_email_send_rate_limit':
        return 'Se enviaron demasiados emails. Espera unos minutos e intenta de nuevo.';
    }
  }
  return mensajeGenerico;
}

/// Pantalla 0 – login/registro con Supabase Auth. Al loguearse con exito no
/// navega a mano: AuthGate (main.dart) escucha el cambio de sesion y muestra
/// la Pantalla 1 automaticamente.
class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final _authService = AuthService();
  final _formKey = GlobalKey<FormState>();
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();

  bool _esRegistro = false;
  bool _cargando = false;
  bool _mostrarPassword = false;
  String? _error;
  String? _mensajeExito;

  // Requisitos de contraseña – solo se exigen al registrarse, no al iniciar
  // sesion (una cuenta vieja puede tener una contraseña mas corta que la
  // regla actual, y no tiene que quedar bloqueada por eso).
  bool get _tieneOchoCaracteres => _passwordController.text.length >= 8;
  bool get _tieneLetra => _passwordController.text.contains(RegExp(r'[A-Za-z]'));
  bool get _tieneNumero => _passwordController.text.contains(RegExp(r'[0-9]'));
  bool get _passwordEsValida => _tieneOchoCaracteres && _tieneLetra && _tieneNumero;

  @override
```

```

void dispose() {
  _emailController.dispose();
  _passwordController.dispose();
  super.dispose();
}

Future<void> _enviar() async {
  if (!_formKey.currentState!.validate()) return;
  setState(() {
    _cargando = true;
    _error = null;
    _mensajeExito = null;
  });
  try {
    if (_esRegistro) {
      final respuesta = await _authService.registrarse(
        email: _emailController.text.trim(),
        password: _passwordController.text,
      );
      // Supabase no tira error si el mail ya existe y ya esta confirmado
      // (para no revelar que mails tienen cuenta) – pero devuelve la lista
      // de "identities" vacia en ese caso, a diferencia de un registro
      // realmente nuevo. Es la unica forma de distinguir ambos casos.
      final yaExistia = respuesta.user?.identities?.isEmpty ?? false;
      setState(() {
        if (yaExistia) {
          _error = 'Ya existe una cuenta registrada con ese email.';
        } else {
          _mensajeExito = 'Te registraste con exito. Revisa tu correo '
            '(y la carpeta de spam) y toca el link de confirmacion antes '
            'de iniciar sesion.';
        }
      });
    } else {
      await _authService.iniciarSesion(
        email: _emailController.text.trim(),
        password: _passwordController.text,
      );
    }
  } catch (e) {
    debugPrint('Error de auth (${_esRegistro ? "registro" : "login"}): $e');
    setState(() {
      _error = _mensajeError(
        e,
        mensajeGenerico: _esRegistro
          ? 'No se pudo completar el registro. Intenta de nuevo.'
          : 'No se pudo iniciar sesion. Intenta de nuevo.',
      );
    });
  } finally {
    if (mounted) setState(() => _cargando = false);
  }
}

/// Manda el mail de "restablecer contraseña". El link del mail no abre la
/// app, abre una pagina web aparte (docs/restablecer.html) donde se pone
/// la contraseña nueva – evita tener que armar deep links por ahora.
Future<void> _recuperarPassword() async {
  final email = _emailController.text.trim();
  if (email.isEmpty || !email.contains('@')) {
    setState(() => _error = 'Escribi tu email arriba antes de tocar este boton.');
```

```
    await _authService.restablecerPassword(email: email);
    setState(() {
      _mensajeExito = 'Te enviamos un mail para restablecer tu contraseña. '
        'Revisa tu correo (y la carpeta de spam).';
    });
  } catch (e) {
    debugPrint('Error al pedir recuperacion de password: $e');
    setState(() {
      _error = _mensajeError(
        e,
        mensajeGenerico: 'No se pudo enviar el mail de recuperacion. Intenta de nuevo.',
      );
    });
  } finally {
    if (mounted) setState(() => _cargando = false);
  }
}

// (build() y _reglaPassword() no van aca por espacio - arman el formulario
// visual: email, contraseña con checklist en vivo, boton de "olvidaste tu
// contraseña", y el boton principal de enviar. Ver el archivo real en
// lib/screens/login_screen.dart para el detalle visual completo.)
}
```

## lib/screens/selector\_campo\_screen.dart (Pantalla 1)

```
import 'package:flutter/material.dart';

import '../models/campo.dart';
import '../services/auth_service.dart';
import '../services/campos_service.dart';
import 'planilla_screen.dart';

/// Pantalla 1 – lista los campos del productor logueado (filtrados por RLS).
/// Si tiene uno solo, entra directo a la planilla; si no tiene ninguno,
/// ofrece crear el primero.
class SelectorCampoScreen extends StatefulWidget {
  const SelectorCampoScreen({super.key});

  @override
  State<SelectorCampoScreen> createState() => _SelectorCampoScreenState();
}

class _SelectorCampoScreenState extends State<SelectorCampoScreen> {
  final _camposService = CamposService();
  final _authService = AuthService();

  late Future<List<Campo>> _futureCampos;

  @override
  void initState() {
    super.initState();
    _futureCampos = _cargarCampos();
  }

  Future<List<Campo>> _cargarCampos() async {
    final campos = await _camposService.traerCampos();
    if (campos.length == 1) {
      // Autoseleccion: si el productor tiene un solo campo, no le hace falta
      // elegir – entra directo a la planilla de ese campo.
      WidgetsBinding.instance.addPostFrameCallback((_) {
        if (!mounted) return;
        Navigator.of(context).pushReplacement(
          MaterialPageRoute(builder: (_) => PlanillaScreen(campo: campos.first)),
        );
      });
    }
    return campos;
  }

  // (el resto de la pantalla dibuja la lista de campos, un boton "+" para
  // crear uno nuevo, y el dialogo con el formulario de nombre/ubicacion.
  // Ver el archivo real para el detalle visual completo.)
}
```

## lib/screens/planilla\_screen.dart (Pantalla 2)

```
import 'package:flutter/material.dart';

import '../models/animal.dart';
import '../models/campo.dart';
import '../services/animales_service.dart';
import '../widgets/animal_card.dart';
import '../widgets/categoria_badge.dart';
import 'conteo_screen.dart';
import 'detalle_animal_screen.dart';
import 'historial_screen.dart';

/// Pantalla 2 – planilla general: lista los animales vivos del campo
/// seleccionado, con búsqueda por rfid_uid y filtro por categoria.
class PlanillaScreen extends StatefulWidget {
  final Campo campo;

  const PlanillaScreen({super.key, required this.campo});

  @override
  State<PlanillaScreen> createState() => _PlanillaScreenState();
}

class _PlanillaScreenState extends State<PlanillaScreen> {
  final _animalesService = AnimalesService();
  late Future<List<Animal>> _futureAnimales;

  String _busqueda = '';
  String? _categoriaFiltro;

  static const _categorias = ['vaca', 'toro', 'ternero', 'ternera', 'vaquillona'];

  @override
  void initState() {
    super.initState();
    _futureAnimales = _animalesService.traerAnimalesDeCampo(widget.campo.id);
  }

  List<Animal> _filtrar(List<Animal> animales) {
    return animales.where((animal) {
      final coincideBusqueda =
        _busqueda.isEmpty || animal.rfidUid.toLowerCase().contains(_busqueda.toLowerCase());
      final coincideCategoria = _categoriaFiltro == null || animal.categoria == _categoriaFiltro;
      return coincideBusqueda && coincideCategoria;
    }).toList();
  }

  // El boton "Iniciar conteo" del final de la pantalla navega a ConteoScreen
  // (Video/Pantalla 3), pasandole el campo actual y la lista de animales ya
  // cargada (para no tener que volver a pedirla).

  // (build() dibuja: barra de búsqueda + dropdown de categoria, la lista con
  // AnimalCard por cada animal, y el boton de iniciar conteo abajo. Ver el
  // archivo real para el detalle visual completo.)
}
```

## lib/screens/conteo\_screen.dart (Pantalla 3 – la más compleja)

```
import 'package:flutter/material.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

import '../models/animal.dart';
import '../models/campo.dart';
import '../models/lectura.dart';
import '../services/conteos_service.dart';
import '../services/lecturas_service.dart';
import '../widgets/animal_card.dart';
import 'detalle_animal_screen.dart';

/// Pantalla 3 – conteo en vivo. Escucha datos_lectura por Supabase Realtime y
/// marca cada animal apenas llega su lectura. Además maneja la fila en
/// `conteos`, que la Raspberry Pi consulta para saber a que conteo activo
/// mandarle las lecturas que va leyendo.
class ConteoScreen extends StatefulWidget {
  final Campo campo;
  final List<Animal> animales;

  const ConteoScreen({super.key, required this.campo, required this.animales});

  @override
  State<ConteoScreen> createState() => _ConteoScreenState();
}

class _ConteoScreenState extends State<ConteoScreen> {
  final _lecturasService = LecturasService();
  final _conteosService = ConteosService();
  final Set<String> _idsDetectados = {};

  RealtimeChannel? _canal;
  String? _conteoId;
  bool _enPausa = false;

  @override
  void initState() {
    super.initState();
    _inicializarConteo();
  }

  Future<void> _inicializarConteo() async {
    // Al entrar a la pantalla: crea (o reanuda) la fila en "conteos", y
    // recién después empieza a escuchar lecturas nuevas por Realtime.
    _conteoId = await _conteosService.iniciarOReanudar(widget.campo.id);
    _empezarAEscuchar();
  }

  void _empezarAEscuchar() {
    _canal = _lecturasService.escucharNuevasLecturas(
      campoId: widget.campo.id,
      alRecibirLectura: _marcarLectura,
    );
  }

  void _marcarLectura(Lectura lectura) {
    // El emparejamiento lectura-animal es por rfid_uid – el texto que lee
    // el lector físico, no por ningún id interno.
    Animal? animalCoincidente;
    for (final animal in widget.animales) {
      if (animal.rfidUid == lectura.rfidUid) {
        animalCoincidente = animal;
        break;
      }
    }
    if (animalCoincidente == null) return;
  }
```

```

        setState(() => _idsDetectados.add(animalCoincidente!.id));
    }

    Future<void> _detener() async {
        if (_canal != null) {
            await _lecturasService.dejarDeEscuchar(_canal!);
            _canal = null;
        }
        if (_conteoId != null) {
            await _conteosService.pausar(_conteoId!);
        }
        if (mounted) setState(() => _enPausa = true);
    }

    Future<void> _reanudar() async {
        if (_conteoId != null) {
            await _conteosService.reanudar(_conteoId!);
        }
        _empezarAEscuchar();
        if (mounted) setState(() => _enPausa = false);
    }

    Future<void> _finalizar() async {
        if (_canal != null) {
            await _lecturasService.dejarDeEscuchar(_canal!);
            _canal = null;
        }
        if (_conteoId != null) {
            await _conteosService.finalizar(_conteoId!);
        }
        if (mounted) Navigator.of(context).pop();
    }

    @override
    void dispose() {
        // Si el usuario sale de la pantalla sin tocar "Detener" ni "Finalizar",
        // igual se deja de escuchar el canal de Realtime (no queda huerfano).
        if (_canal != null) {
            _lecturasService.dejarDeEscuchar(_canal!);
        }
        super.dispose();
    }

    // (build() dibuja: contador "X / Y detectados" arriba, la lista de
    // animales con un tilde verde si ya fueron detectados, y los botones
    // Detener/Reanudar + Finalizar abajo. Ver el archivo real para el detalle
    // visual completo.)
}

```

```

import 'package:flutter/material.dart';

import '../models/animal.dart';
import '../theme/app_theme.dart';
import '../widgets/categoria_badge.dart';
import 'fertilidad_screen.dart';
import 'observaciones_screen.dart';
import 'vacunas_screen.dart';

/// Pantalla 4 – ficha de un animal: ID, categoria, y tres botones (con
/// icono) que llevan a la pantalla de edicion de cada dato (fertilidad,
/// vacunas, observaciones) – son datos que el lector RFID no puede leer
/// solo, los carga el productor.
class DetalleAnimalScreen extends StatelessWidget {
  final Animal animal;

  const DetalleAnimalScreen({super.key, required this.animal});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(animal.rfidUid)),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Center(child: CategoriaBadge(categoria: animal.categoria)),
            const SizedBox(height: 24),
            Row(
              children: [
                _BotonIcono(
                  icono: Icons.favorite,
                  etiqueta: 'Fertilidad',
                  onTap: () => Navigator.of(context).push(
                    MaterialPageRoute(builder: (_) => FertilidadScreen(animal: animal)),
                  ),
                const SizedBox(width: 12),
                _BotonIcono(
                  icono: Icons.vaccines,
                  etiqueta: 'Vacunas',
                  onTap: () => Navigator.of(context).push(
                    MaterialPageRoute(builder: (_) => VacunasScreen(animal: animal)),
                  ),
                const SizedBox(width: 12),
                _BotonIcono(
                  icono: Icons.visibility,
                  etiqueta: 'Observaciones',
                  onTap: () => Navigator.of(context).push(
                    MaterialPageRoute(builder: (_) => ObservacionesScreen(animal: animal)),
                  ),
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}

/// Boton cuadrado con icono arriba y etiqueta abajo, usado para los 3
/// accesos de la Pantalla 4. Queda privado a este archivo (el guion bajo).

```

```

class _BotonIcono extends StatelessWidget {
  final IconData icono;
  final String etiqueta;
  final VoidCallback onTap;

  const _BotonIcono({
    required this.icono,
    required this.etiqueta,
    required this.onTap,
  });

  @override
  Widget build(BuildContext context) {
    return Expanded(
      child: InkWell(
        onTap: onTap,
        borderRadius: BorderRadius.circular(12),
        child: Container(
          padding: const EdgeInsets.symmetric(vertical: 16),
          decoration: BoxDecoration(
            border: Border.all(color: AppColors.marronPrincipal),
            borderRadius: BorderRadius.circular(12),
          ),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
              Icon(icono, color: AppColors.marronPrincipal, size: 32),
              const SizedBox(height: 8),
              Text(
                etiqueta,
                textAlign: TextAlign.center,
                style: const TextStyle(color: AppColors.marronPrincipal, fontSize: 12),
              ),
            ],
          ),
        ),
      ),
    );
  }
}

```

## lib/screens/fertilidad\_screen.dart

```
import 'package:flutter/material.dart';

import '../models/animal.dart';
import '../services/animales_service.dart';

/// Pantalla para editar la fertilidad de un animal – se llega acá tocando el
/// botón "Fertilidad" en la ficha del animal (Pantalla 4).
class FertilidadScreen extends StatefulWidget {
  final Animal animal;

  const FertilidadScreen({super.key, required this.animal});

  @override
  State<FertilidadScreen> createState() => _FertilidadScreenState();
}

class _FertilidadScreenState extends State<FertilidadScreen> {
  final _animalesService = AnimalesService();
  String? _fertilidadSeleccionada;
  bool _guardando = false;

  @override
  void initState() {
    super.initState();
    _fertilidadSeleccionada = widget.animal.fertilidad;
  }

  Future<void> _guardar() async {
    setState(() => _guardando = true);
    try {
      await _animalesService.actualizarAnimal(
        animalId: widget.animal.id,
        observaciones: widget.animal.observaciones,
        fertilidad: _fertilidadSeleccionada,
      );
      if (mounted) Navigator.of(context).pop();
    } finally {
      if (mounted) setState(() => _guardando = false);
    }
  }

  // (build() dibuja un dropdown con buena/regular/mala y el boton Guardar.)
}
```

## lib/screens/vacunas\_screen.dart

```
import 'package:flutter/material.dart';

import '../models/animal.dart';
import '../models/vacuna.dart';
import '../services/vacunas_service.dart';

/// Pantalla para ver la ultima vacuna y agregar una nueva – se llega acá
/// tocando el botón "Vacunas" en la ficha del animal (Pantalla 4).
class VacunasScreen extends StatefulWidget {
  final Animal animal;

  const VacunasScreen({super.key, required this.animal});

  @override
  State<VacunasScreen> createState() => _VacunasScreenState();
}

class _VacunasScreenState extends State<VacunasScreen> {
  final _vacunasService = VacunasService();
  final _nombreVacunaController = TextEditingController();
  late Future<Vacuna?> _futureVacuna;
  DateTime? _fechaVacunaSeleccionada;

  @override
  void initState() {
    super.initState();
    _futureVacuna = _vacunasService.traerUltimaVacuna(widget.animal.id);
  }

  Future<void> _agregarVacuna() async {
    if (_nombreVacunaController.text.trim().isEmpty) return;
    await _vacunasService.agregarVacuna(
      idAnimal: widget.animal.id,
      nombreVacuna: _nombreVacunaController.text.trim(),
      fechaVacuna: _fechaVacunaSeleccionada,
    );
    _nombreVacunaController.clear();
    setState(() {
      _fechaVacunaSeleccionada = null;
      _futureVacuna = _vacunasService.traerUltimaVacuna(widget.animal.id);
    });
    if (mounted) {
      ScaffoldMessenger.of(context)
        .showSnackBar(const SnackBar(content: Text('Vacuna agregada.')));
    }
  }

  // (build()) dibuja la ultima vacuna, un campo de texto + selector de fecha,
  // y el boton "Agregar vacuna".
}
```

## lib/screens/observaciones\_screen.dart

```
import 'package:flutter/material.dart';

import '../models/animal.dart';
import '../services/animales_service.dart';

/// Pantalla para editar las observaciones de un animal – se llega acá
/// tocando el botón "Observaciones" en la ficha del animal (Pantalla 4).
class ObservacionesScreen extends StatefulWidget {
  final Animal animal;

  const ObservacionesScreen({super.key, required this.animal});

  @override
  State<ObservacionesScreen> createState() => _ObservacionesScreenState();
}

class _ObservacionesScreenState extends State<ObservacionesScreen> {
  final _animalesService = AnimalesService();
  final _observacionesController = TextEditingController();
  bool _guardando = false;

  @override
  void initState() {
    super.initState();
    _observacionesController.text = widget.animal.observaciones ?? '';
  }

  Future<void> _guardar() async {
    setState(() => _guardando = true);
    try {
      final observaciones = _observacionesController.text.trim();
      await _animalesService.actualizarAnimal(
        animalId: widget.animal.id,
        observaciones: observaciones.isEmpty ? null : observaciones,
        fertilidad: widget.animal.fertilidad,
      );
      if (mounted) Navigator.of(context).pop();
    } finally {
      if (mounted) setState(() => _guardando = false);
    }
  }

  // (build()) dibuja un campo de texto multilinea y el boton Guardar.
}
```

## lib/screens/historial\_screen.dart (Pantalla 5)

```
import 'package:flutter/material.dart';

import '../models/campo.dart';
import '../models/lectura.dart';
import '../services/lecturas_service.dart';

/// Pantalla 5 – últimas lecturas del campo actual, mas reciente primero.
class HistorialScreen extends StatefulWidget {
  final Campo campo;

  const HistorialScreen({super.key, required this.campo});

  @override
  State<HistorialScreen> createState() => _HistorialScreenState();
}

class _HistorialScreenState extends State<HistorialScreen> {
  final _lecturasService = LecturasService();
  late Future<List<Lectura>> _futureLecturas;

  @override
  void initState() {
    super.initState();
    _futureLecturas = _lecturasService.traerHistorial(widget.campo.id);
  }

  String _formatearFecha(DateTime fecha) {
    final f = fecha.toLocal();
    final dia = f.day.toString().padLeft(2, '0');
    final mes = f.month.toString().padLeft(2, '0');
    final hora = f.hour.toString().padLeft(2, '0');
    final minuto = f.minute.toString().padLeft(2, '0');
    return '$dia/$mes/${f.year} $hora:$minuto';
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Historial de lecturas')),
      body: FutureBuilder<List<Lectura>>(
        future: _futureLecturas,
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(child: CircularProgressIndicator());
          }
          final lecturas = snapshot.data ?? [];
          if (lecturas.isEmpty) {
            return const Center(child: Text('Todavía no hay lecturas registradas.'));
          }
          return ListView.builder(
            itemCount: lecturas.length,
            itemBuilder: (context, indice) {
              final lectura = lecturas[indice];
              return ListTile(
                leading: const Icon(Icons.nfc),
                title: Text(lectura.rfidUid),
                subtitle: Text(_formatearFecha(lectura.fechaHora)),
              );
            },
          );
        },
      ),
    );
  }
}
```